

BlastShield

AI Pre-Execution Impact Analysis for TrueForge Agents

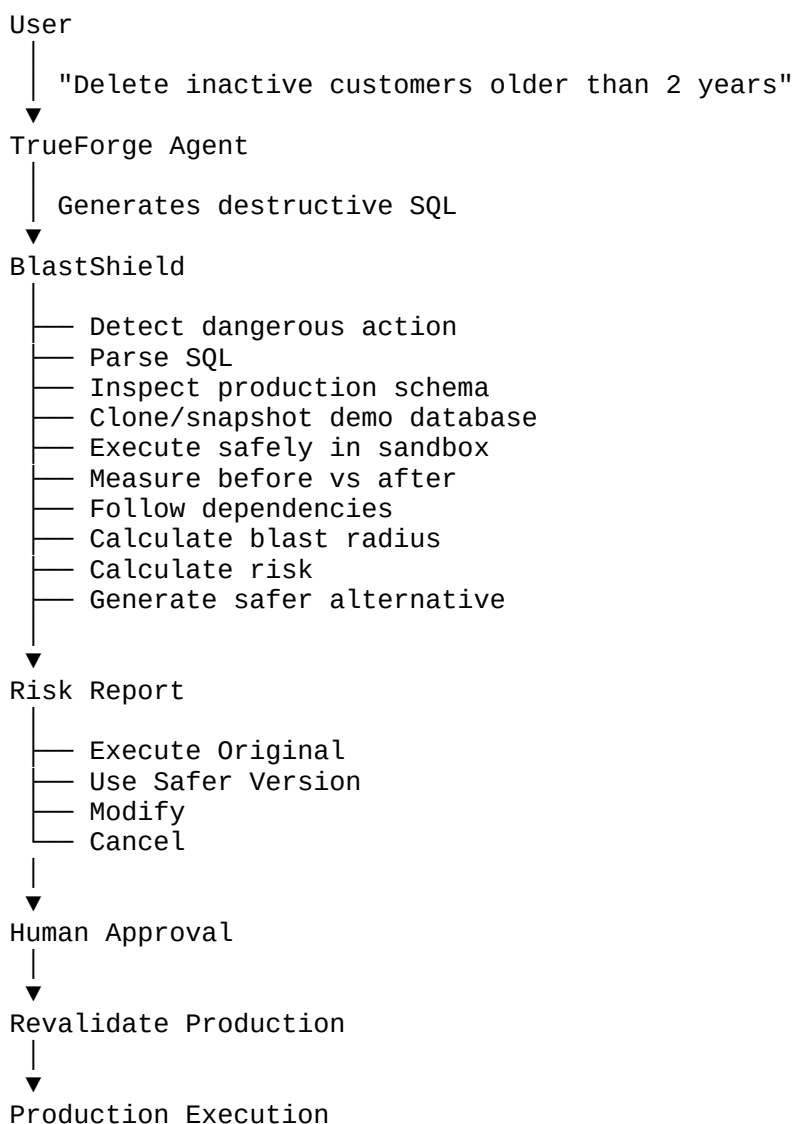
One-line product

BlastShield intercepts dangerous AI-agent actions, simulates them safely, calculates their blast radius, explains the consequences, recommends safer actions, and asks for human approval before production execution.

1. What We Are Actually Building

We are **not** building a generic sandbox system.

We are building this:



2. Four-Day MVP Scope

Must build

Only support:

PostgreSQL
+
TrueForge
+
MCP
+
BlastShield

Dangerous operations:

DELETE
UPDATE
TRUNCATE
DROP TABLE

Safe operations:

SELECT

For the hackathon demo, DELETE should be the most polished operation.

3. What We Should NOT Build

Do not build these during the four days:

✗ AWS integration
✗ Kubernetes integration
✗ Gmail integration
✗ GitHub protection
✗ Multiple database engines
✗ Complete authentication system
✗ Multi-tenant SaaS
✗ Production-grade database replication
✗ Kubernetes deployment
✗ Complicated machine-learning risk model
✗ Perfect universal SQL support

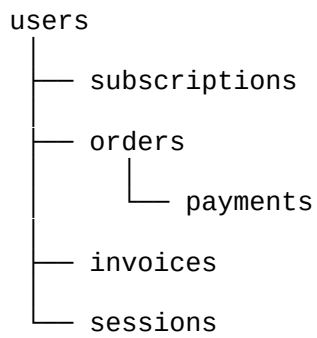
They belong on the:

Future Roadmap

not in the MVP.

4. Demo Database

Build a fake but realistic SaaS production database.



Example tables:

users

id

name

email

status

last_login

created_at

deleted_at

subscriptions

id

user_id

status

monthly_price

started_at

orders

id

user_id

amount

status

created_at

payments

id

order_id

amount

status

invoices

id

user_id

amount

status

sessions

id

user_id

last_active

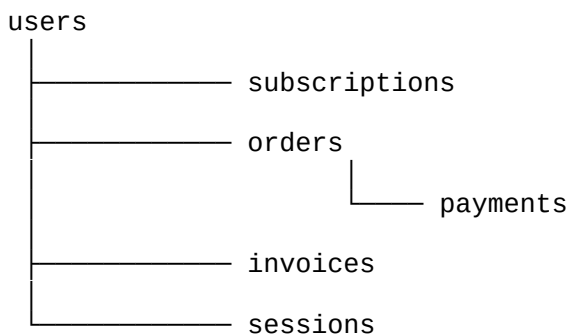
Seed approximately:

Users	50,000
Orders	100,000
Subscriptions	15,000
Payments	90,000
Invoices	30,000
Sessions	120,000

The numbers make the demo visually impressive.

5. Important Database Relationships

Example:



Create some foreign keys using:

ON DELETE CASCADE

That allows BlastShield to demonstrate a real hidden consequence.

Example:

DELETE 12,481 users

does not mean:

12,481 records affected

It might mean:

12,481	users
21,003	orders
18,201	payments
347	subscriptions
5,102	invoices
9,682	sessions

6. TrueForge Agent

Create one agent:

BlastShield Database Operator

Its responsibility:

User request

↓

Understand request

↓

Generate SQL

↓

Never directly execute destructive SQL

↓

Send destructive operation through BlastShield

The agent should have access to safe database tools.

Possible MCP tools:

`get_schema()`

`run_read_query(sql)`

`request_database_change(sql)`

`execute_approved_change(change_id)`

This is important.

Do NOT expose:

`run_any_sql(sql)`

directly to the agent.

Otherwise the agent could bypass BlastShield.

7. MCP Architecture

Your custom PostgreSQL MCP server should expose controlled tools.

Tool 1

`get_schema()`

Returns:

tables

columns

primary keys

foreign keys

constraints

cascade rules

Tool 2

`run_read_query(sql)`

Allowed:

SELECT
EXPLAIN

Blocked:

DELETE
UPDATE
DROP
TRUNCATE
ALTER
INSERT

Tool 3

`request_database_change(sql)`

This is the core tool.

Example:

```
request_database_change(  
    "DELETE FROM users  
    WHERE last_login < NOW() - INTERVAL '2 years'"  
)
```

Instead of executing it, the tool returns:

```
{  
  "analysis_id": "abc123",  
  "status": "requires_review"  
}
```

BlastShield then performs analysis.

Tool 4

`execute_approved_change(analysis_id)`

This tool must only work when:

`analysis.status == APPROVED`

Otherwise:

Execution denied.

8. Core Backend Architecture

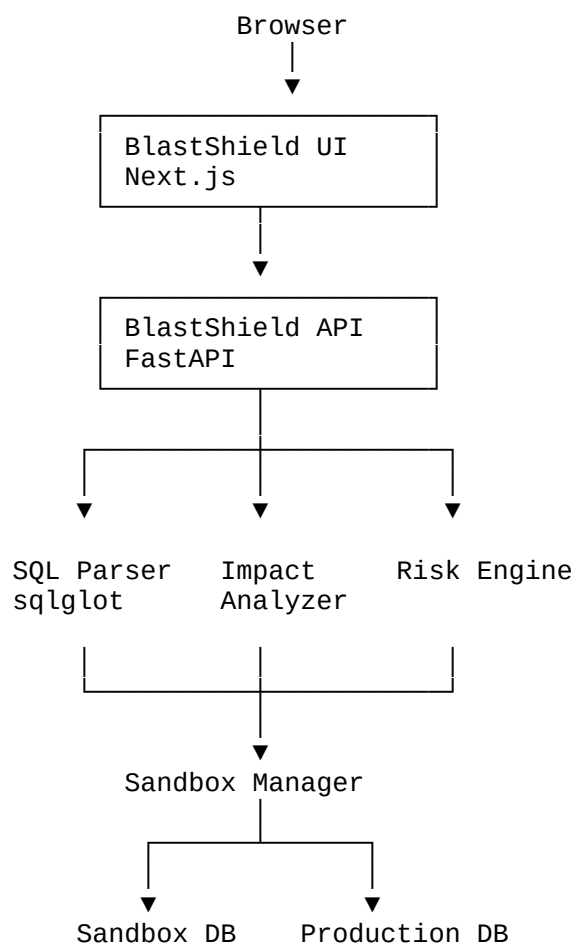
Use:

Python
FastAPI
PostgreSQL
SQLAlchemy
sqlglot
Docker
TrueForge
MCP

Frontend:

Next.js
TypeScript
Tailwind
shadcn/ui

System:



TrueForge sits above this workflow:

User
↓
TrueForge

↓
MCP
↓
BlastShield API
↓
Sandbox analysis
↓
TrueForge approval
↓
Production

9. Backend Modules

Structure the backend approximately like:

```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── analysis.py
│   │   └── execution.py
│   ├── services/
│   │   ├── sql_parser.py
│   │   ├── schema_analyzer.py
│   │   ├── impact_analyzer.py
│   │   ├── sandbox_manager.py
│   │   ├── risk_engine.py
│   │   ├── alternative_generator.py
│   │   └── execution_service.py
│   ├── models/
│   │   ├── analysis.py
│   │   └── impact.py
│   └── database/
│       └── connection.py
├── mcp/
│   └── server.py
├── scripts/
│   ├── seed_database.py
│   └── create_snapshot.sh
└── tests/
```

Frontend:

```
frontend/
├── app/
│   ├── page.tsx
│   └── analysis/[id]/page.tsx
├── components/
│   └── AgentChat.tsx
```


- ActionTimeline.tsx
- RiskBadge.tsx
- ImpactCard.tsx
- BlastRadiusGraph.tsx
- SqlViewer.tsx
- AlternativeComparison.tsx
- ApprovalBar.tsx

10. SQL Parser

Use:

sqlglot

Do not use regex as your primary SQL parser.

Input:

```
DELETE FROM users
WHERE last_login < NOW() - INTERVAL '2 years';
```

Extract:

```
{
  "operation": "DELETE",
  "table": "users",
  "condition": "last_login < ..."
}
```

11. Action Classification

Classify commands.

SELECT

↓

SAFE

INSERT

↓

MEDIUM

UPDATE

↓

POTENTIALLY DANGEROUS

DELETE

↓

HIGH RISK

TRUNCATE

↓

CRITICAL

DROP

↓

CRITICAL

This should be deterministic code.

Do not ask the LLM:

Is DELETE dangerous?

Your backend should already know.

12. Sandbox Design

For the hackathon, do not attempt sophisticated production cloning.

Have:

blastshield_prod

blastshield_sandbox

Both can run in PostgreSQL through Docker.

Initial setup:

Seed production database

↓

Create clean snapshot

↓

Restore snapshot into sandbox

Whenever analysis begins:

Reset sandbox

↓

Restore clean state

↓

Execute proposed SQL there

↓

Measure results

Possible implementation:

pg_dump

+

pg_restore

or maintain a deterministic seed script.

The judges care about the concept working reliably more than having enterprise-grade replication.

13. Before/After Simulation

This is one of the most important features.

Before executing:

users	50,000
orders	100,000
subscriptions	15,000

Execute destructive SQL in sandbox.

After:

users	37,519
orders	78,997
subscriptions	14,653

Calculate:

users:
50,000 → 37,519

difference:
12,481

Do the same for related tables.

14. Dependency Analyzer

Query PostgreSQL metadata.

Find:

foreign keys
ON DELETE CASCADE
ON UPDATE CASCADE
dependent tables

Build:

users

```
├── subscriptions
├── orders
│   └── payments
├── invoices
└── sessions
```

For four days, limit recursive dependency analysis to something like:

2-3 levels

You do not need arbitrary graph traversal across a huge enterprise database.

15. Business Impact Analyzer

This is what makes the project better than:

"12,481 rows affected."

Example query:

```
SELECT COUNT(*)
FROM subscriptions
WHERE status = 'active'
AND user_id IN (...);
```

Return:

347 active subscriptions affected

Then:

```
SELECT SUM(monthly_price)
FROM subscriptions
...
```

Return:

MRR at risk: \$6,116

Then:

Estimated ARR at risk:
\$73,392

Now the UI says:

347 paying customers affected

\$73.4K estimated ARR at risk

That is much more powerful than simply saying:

DELETE affects 12,481 rows.

16. Risk Engine

Do not use a complicated AI risk model.

Make the scoring deterministic.

Example:

Risk Score =

Operation severity	0-25
Rows affected	0-25
Cascade impact	0-20
Business-critical data	0-20
Reversibility	0-10

	100

Example:

DELETE	20
12,481 users	22
3 cascades	15
347 active subscriptions	20
Hard rollback	8

TOTAL	85

Result:

85 / 100

▣ CRITICAL

Risk levels:

0-24

LOW

25-49

MEDIUM

50-74

HIGH

75-100

CRITICAL

17. Safer Alternative Generator

Example original:

```
DELETE FROM users
WHERE last_login < NOW() - INTERVAL '2 years';
```

Safer option:

```
UPDATE users
SET deleted_at = NOW()
WHERE last_login < NOW() - INTERVAL '2 years';
```

Then BlastShield analyzes the safer command too.

UI:

Original

▣ Risk 85
48,615 affected records
Hard to recover

VS

Recommended

▣ Risk 34
12,481 rows updated
0 cascading deletions
Recoverable

This comparison will look excellent during the demo.

18. Production Revalidation

This can become one of your most technically impressive features.

Suppose analysis says:

12,481 users affected.

Human waits 30 seconds.

Production data changes.

Before actual execution, BlastShield runs the preview again.

Approved:

12,481 users

Current:

12,912 users

If the difference exceeds a threshold:

⚠ PRODUCTION STATE CHANGED

Previous approval invalidated.

Re-analysis required.

This prevents stale approvals.

For the hackathon, implement a basic equality check.

```
if current_affected_rows != approved_affected_rows:  
    reject_execution()
```

Excellent demo feature.

19. BlastShield Metadata Tables

You need somewhere to store analysis.

analyses

id
session_id
original_sql
operation_type
target_table
risk_score
risk_level
direct_rows
indirect_rows
status
safer_sql
created_at
approved_at
executed_at

Statuses:

ANALYZING
WAITING_APPROVAL
APPROVED
REJECTED
EXECUTED
INVALIDATED

impact_items

id
analysis_id
table_name
relationship
affected_rows
impact_type

Example:

orders
CASCADE
21,003
DEPENDENT

audit_events

id
analysis_id
event_type
payload
created_at

Events:

ACTION_INTERCEPTED
SANDBOX_CREATED
SIMULATION_COMPLETED
RISK_CALCULATED
USER_APPROVED
EXECUTION_STARTED
EXECUTION_COMPLETED

20. API Requirements

You don't need 25 endpoints.

Build approximately these.

Start analysis

POST /api/analysis

Request:

```
{  
  "sql": "DELETE FROM users ..."  
}
```

Response:

```
{  
  "analysis_id": "abc123"  
}
```

Get analysis

GET /api/analysis/abc123

Returns:

```
{  
  "risk": 85,  
  "level": "critical",  
  "direct_rows": 12481,  
  "indirect_rows": 36134  
}
```

Approve

POST /api/analysis/abc123/approve

Execute

POST /api/analysis/abc123/execute

Before execution:

Revalidate

↓

If same

↓

Execute

21. UI Strategy

Your UI should **not look like another ChatGPT clone**.

Make it look like:

AI Agent

+

Production Operations Console

+

Security Tool

Think:

Vercel

Linear

Datadog

Sentry

GitHub

style rather than a generic AI chatbot.

22. Main Desktop Layout

Design for:

1440 × 1024

Use three areas:

BlastShield			PROD • Agent Connected		
Sidebar		AGENT CHAT		IMPACT PANEL	
Workspace		User prompt		Risk	
Sessions		Agent reasoning/events		Impact	
History				Dependencies	
Settings				Recommendation	

Suggested proportions:

Sidebar

220px

Agent workspace

~600px

Impact panel

remaining space

23. Figma File Structure

Create four Figma pages.

01 — Foundations

02 — Components

03 — Screens

04 — Prototype

24. Figma Page 01 — Foundations

Define typography.

Example:

Display
32 / 40

Heading
24 / 32

Section
18 / 26

Body
14 / 20

Small
12 / 16

Code
13 / 20

Use:

Inter

for general UI.

Use a monospace font for:

SQL
IDs
logs
tool calls
metrics

Create variables for:

background
surface
surface-secondary
border
text-primary
text-secondary

risk-low
risk-medium
risk-high
risk-critical
success

Also spacing variables:

4
8
12
16
24
32
48

25. Figma Page 02 — Components

Create reusable components.

Buttons

Variants:

Primary
Secondary
Danger
Ghost

States:

Default
Hover
Disabled
Loading

Risk Badge

Variants:

LOW
MEDIUM
HIGH
CRITICAL

Example:

- CRITICAL
-

Metric Card

Example:

DIRECT IMPACT

12,481
Users

+31% risk

Tool Event

Example:

- ✓ Schema inspected
 - ✓ Sandbox created
 - ✓ Simulation complete
 - ✓ Dependencies traced
-

SQL Block

```
DELETE FROM users
WHERE last_login < ...
```

Include:

Copy button
language label
operation badge

Approval Bar

[Cancel]

[Modify]

[Use Safe Version]

[Execute Anyway]

This should always remain visible when reviewing a dangerous operation.

26. Screen 1 — Agent Workspace

Before dangerous action:

James

Delete customers inactive for more
than two years.

BlastShield Agent

I'll analyze the requested change
before allowing it to reach production.

Generated Action

DELETE FROM users
WHERE ...

Analyzing...

Right side:

Action Analysis

Waiting for proposed action...

27. Screen 2 — Simulation State

This is important for perceived intelligence.

Show a timeline:

Analyzing production action

✓ SQL parsed

✓ Target table detected

- ✓ Schema loaded
- ✓ Sandbox restored
- Running simulation
 - Following foreign keys
 - Calculating business impact
 - Generating safer alternative

Don't simply show:

Loading...

Show what the system is doing.

That makes your product feel sophisticated.

28. Screen 3 — Risk Report

This is the most important screen.

Top:

BLAST RADIUS

▣ CRITICAL

85 / 100

Metrics:

Users 12,481

Dependencies 36,134

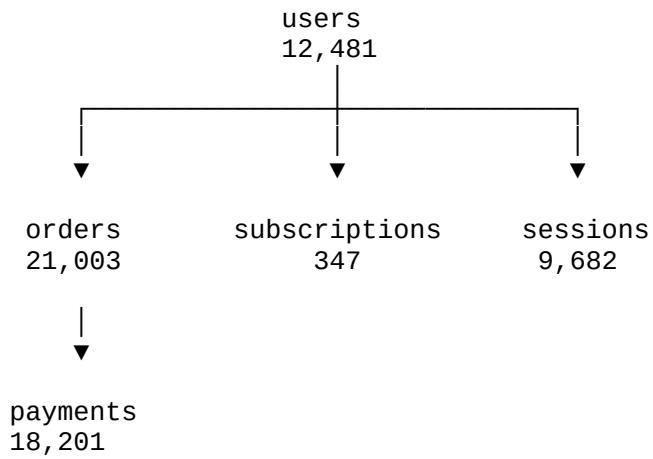
Cascades 3

ARR at Risk \$73.4K

Below:

Blast Radius

Display:



This is your visual centerpiece.

29. Screen 4 — Original vs Safer

Use two large cards.

ORIGINAL

DELETE users...

Risk

▣ 85

Affected

48,615

Rollback

Difficult

versus:

RECOMMENDED

Soft delete users...

Risk

▣ 34

Affected

12,481

Rollback

Easy

Bottom:

30. Screen 5 — Execution

After approval:

Final production validation

- ✓ SQL unchanged
- ✓ Affected rows unchanged
- ✓ Dependency count unchanged
- ✓ Approval valid

Executing...

Then:

- ✓ Production change completed

12,481 users soft-deleted

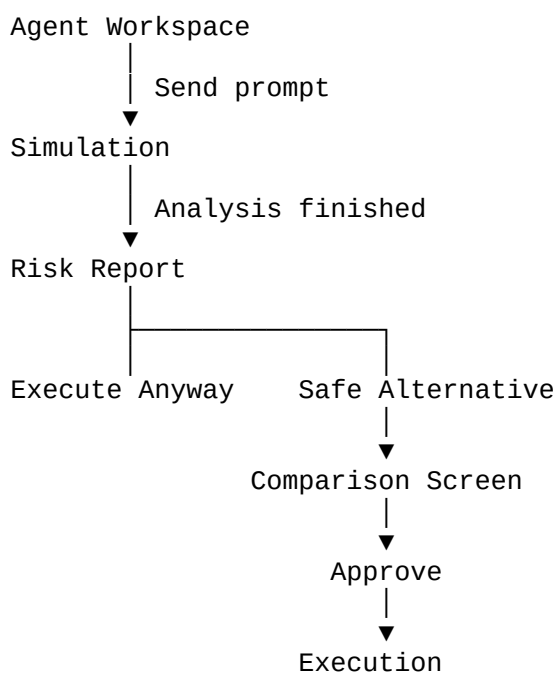
0 cascading records deleted

Audit ID

BS-2026-08321

31. Figma Prototype Flow

Wire screens:



Use Smart Animate only where useful.

Do not spend six hours creating animations.

32. Best UI Interaction

The most memorable interaction should be:

User clicks:

"Use Safer Alternative"

Risk instantly changes:

85

▣ CRITICAL

to:

34

▣ MEDIUM

and:

48,615 records

becomes:

12,481 records

with:

Cascading deletions

3 → 0

That will communicate your value immediately.

33. Frontend Implementation

Use:

Next.js
TypeScript
Tailwind
shadcn/ui

Do not code every component from scratch.

Use shadcn for:

buttons
cards
dialogs

tabs
badges
tooltips
drawers
tables

Custom build only:

BlastRadiusGraph
RiskGauge
ImpactTimeline
OriginalVsSafeComparison

34. Dependency Graph

Do not spend one entire day making a complicated graph engine.

You can use:

React Flow

Example nodes:

Users
12,481

Connected to:

Orders
Subscriptions
Sessions
Invoices

Edges can say:

CASCADE
FK
DEPENDENCY

This gives you a very strong visual with relatively little work.

35. Main User Journey

Your complete hackathon user story should be only this:

1.

User asks agent:

"Delete inactive customers older than 2 years."

2.

TrueForge generates DELETE.

3.

BlastShield intercepts.

4.

Sandbox starts.

5.

SQL executes safely.

6.

BlastShield measures impact.

7.

Risk report appears.

8.

User discovers:

12,481 users

347 paying customers

48K total records

\$73K ARR risk

9.

BlastShield suggests soft delete.

10.

User chooses safer version.

11.

BlastShield re-analyzes.

12.

Risk drops.

13.

User approves.

14.

Production is revalidated.

15.

Safe operation executes.

That is your entire demo.

Do not create five demo scenarios.

Perfect this one.

36. Four-Day Development Plan

DAY 1 — Make the Backbone Work

Goal

By the end of Day 1:

Prompt

↓

TrueForge

↓

MCP

↓

SQL detected

↓

BlastShield API

must work.

Morning

Build repository.

/frontend

/backend

/mcp

/docker

Create Docker Compose:

Postgres production

Postgres sandbox

Backend

Create database schema.

Seed data.

Afternoon

Create MCP server.

Implement:

`get_schema()`

`run_read_query()`

`request_database_change()`

Connect MCP to TrueForge.

Test:

"How many users exist?"

Then:

"Delete inactive users."

Dangerous SQL must reach:

`request_database_change()`

not production.

Evening

Implement:

SQL Parser
operation detector
table detector

Target result:

```
{
  "operation": "DELETE",
  "table": "users",
  "dangerous": true
}
```

DAY 1 DEFINITION OF DONE

You can type:

Delete inactive users.

and see:

 BlastShield intercepted DELETE on users.

Nothing else matters until this works.

DAY 2 — Build the Intelligence

Goal

Produce a real risk report.

Morning

Implement sandbox reset.

```
production snapshot
      ↓
sandbox
```

Execute destructive SQL only against sandbox.

Midday

Implement:

```
before counts
after counts
difference
```

Afternoon

Implement schema dependency discovery.

Detect:

```
users → orders
users → subscriptions
users → sessions
```

Calculate related records.

Evening

Implement deterministic risk engine.

Output:

```
{
  "risk_score": 85,
  "risk_level": "CRITICAL",
  "direct_rows": 12481,
  "indirect_rows": 36134,
  "active_subscriptions": 347
}
```

DAY 2 DEFINITION OF DONE

Calling:

```
request_database_change()
```

must return an actual blast-radius analysis.

No fancy frontend required yet.

DAY 3 — Build the Wow Factor

Goal

Make judges understand BlastShield instantly.

Morning

Implement Figma final screens.

Only polish:

- Workspace
- Risk Report
- Comparison
- Execution

Midday

Build frontend.

Prioritize:

- Risk score
- Metric cards
- Dependency graph
- SQL viewer
- Approval buttons

Afternoon

Connect frontend to FastAPI.

Risk report must use REAL API values.

Not hardcoded numbers.

Evening

Implement safer alternative.

Implement:

Use Safe Alternative

Re-run risk analysis.

Show:

85 → 34

DAY 3 DEFINITION OF DONE

You can perform the entire demo visually except final production execution.

DAY 4 — Safety + Polish + Pitch

Morning

Implement production revalidation.

Analysis

↓

Approval

↓

Recalculate

↓

Compare

↓

Execute

Midday

Implement production execution.

Add audit logging.

Test everything.

Afternoon

Fix UI.

Add:

loading states

empty states

error states

tool timeline

success animation

Do not add new major features.

Evening

Demo rehearsal.

Run the demo at least:

10 times

without touching code.

Prepare fallback:

screen recording

in case internet/LLM/API fails.

DAY 4 DEFINITION OF DONE

You can deliver the entire project story in under five minutes without debugging anything live.

37. Priority Matrix

P0 — Must Work

TrueForge agent

PostgreSQL MCP

Dangerous action interception

SQL parser

Sandbox

Affected row calculation

Dependency analysis

Risk score

Risk report UI

Human approval

Safe alternative

If any P0 feature is broken, stop adding new features.

P1 — Strong Advantage

Business impact

ARR at risk

Dependency visualization

Before/after comparison

Production revalidation

Audit trail

P2 — Only If Everything Else Works

Fancy animations

Multiple agents

GitHub support

Kubernetes example

AI-generated risk explanations

Complete dashboard

User accounts

38. Team Allocation

If you have three people:

Person 1

TrueForge

MCP

Backend

SQL parsing

Person 2

Sandbox

PostgreSQL

Impact analysis

Risk engine

Person 3

Figma

Frontend

React Flow

Demo/presentation

Everyone integrates together every evening.

Do not wait until Day 4 to merge.

39. If You Are Mostly Building It Yourself

Then simplify further.

Build:

DELETE only

Users only as root table

4 dependent tables

One safer alternative

One demo prompt

That can still be an excellent project.

Working depth beats broken breadth.

40. Judge Demo

Do not begin the presentation by explaining architecture for three minutes.

Start with the disaster.

Say:

Imagine your AI agent has production database access.

Then type:

Delete every customer inactive for more than two years.

Agent generates:

```
DELETE FROM users
WHERE last_login < NOW() - INTERVAL '2 years';
```

Then immediately:

 BLASTSHIELD INTERCEPTED

Show scanning.

Then reveal:

 CRITICAL

Risk Score

85 / 100

Direct impact

12,481 users

Hidden impact

36,134 dependent records

347 paying customers

\$73,400 ARR at risk

Rollback

Difficult

Then say:

A normal approval system asks whether I'm allowed to run this SQL. BlastShield tells me what approving it actually means.

Click:

Use Safer Alternative

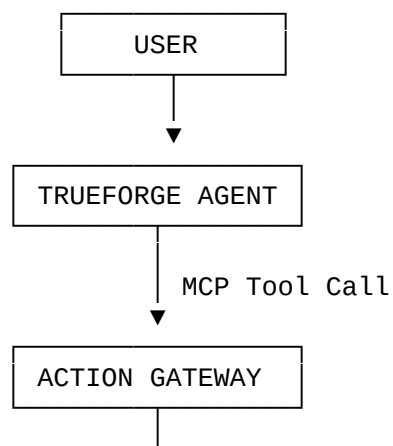
Risk becomes:

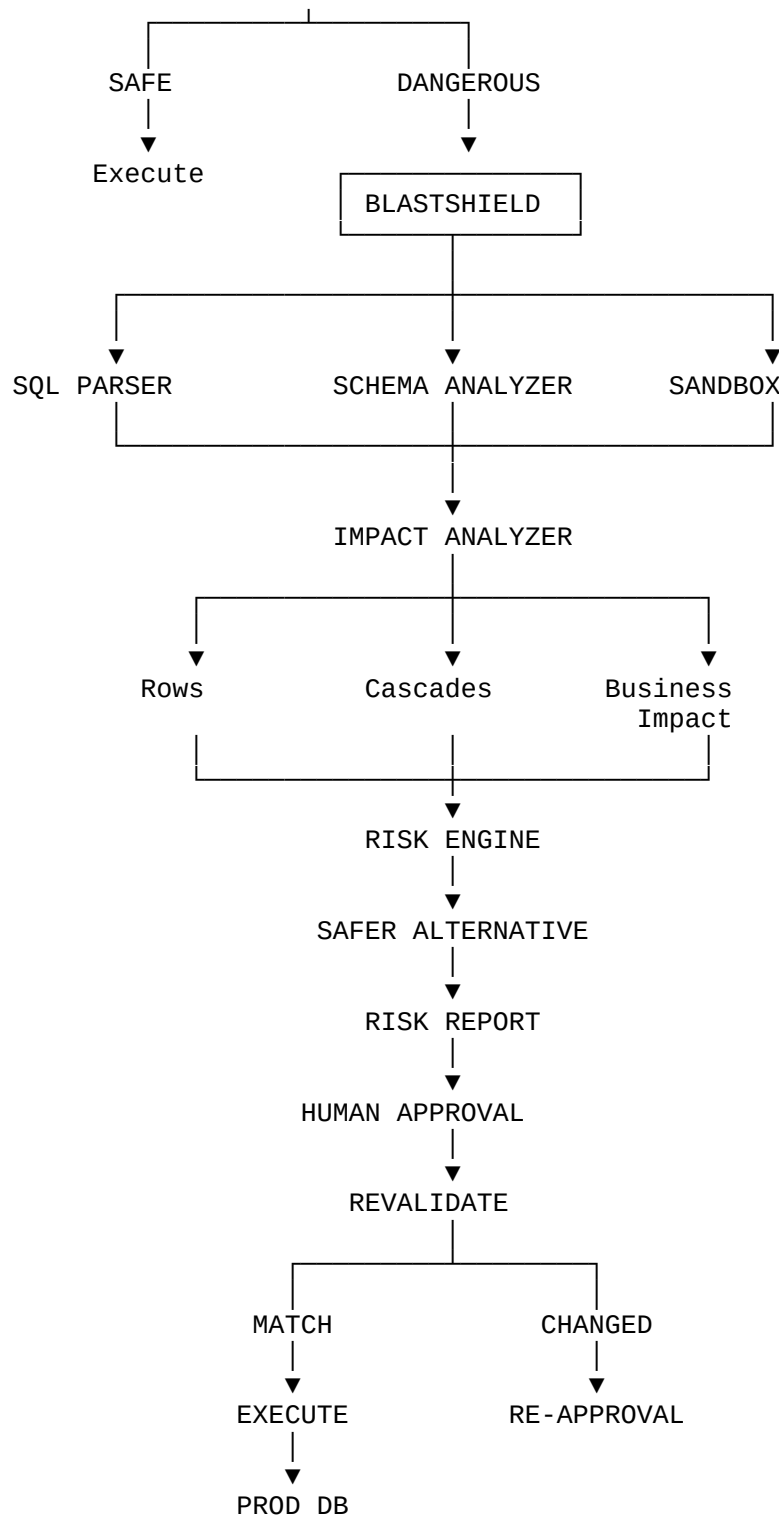
● 34 / 100

Then approve and execute.

That is your wow moment.

41. The System Design in One Diagram





42. Most Important Engineering Principle

The LLM should **propose**.

BlastShield's deterministic backend should **verify**.

The human should **authorize**.

The production tool should **enforce**.

LLM



Proposal

BlastShield



Evidence

Human



Decision

Execution Layer



Enforcement

Do not rely on the same LLM that generated the dangerous operation to simply say:

Don't worry, this operation is safe.

Your safety boundary should come from actual database inspection and simulation.

43. Final MVP Requirement

By the hackathon deadline, you only need to prove this:

AI intended to do X.

Without BlastShield:
X would reach production.

With BlastShield:
we discovered X would cause Y and Z.

BlastShield showed the consequences.

BlastShield proposed a safer action.

Human chose the safer action.

Production received only the approved operation.

If that entire chain genuinely works, you have a strong hackathon submission.

Final Product Positioning

TrueForge gives agents the ability to act safely within a controlled harness.

BlastShield adds consequence-aware execution: before a dangerous agent action reaches production, it proves what that action is likely to do.

BlastShield — Know the damage before the agent does it.